

Path planning

Marko Guberina

September 25, 2025

1 Path planning

We're using a path planner based on dynamical systems from Albin Dahlin. The theory is amazing and super interesting, the results are phenomenal. However, the purpose of the project, we only need to know how it works to tune it. From the standpoint of control, it already works well enough to be tested in simulation.

Another important point is that replanning is real-time (about 200Hz). Thus, we can count on the beginning of the path to always coincide with the origin of whatever it is we're planning for. This assumption is used in path-following control.

1.1 [TODO: expand a bit] Planner

The path is a limited-horizon orbit of a dynamical system describing the flow towards the goal:

$$\dot{r} = M(r, \mathcal{O})(r^g - r)$$

where r is the current robot position, r^g the desired goal, $M(\cdot, \cdot)$ a matrix that describes the attractive dynamics, and $\mathcal{O}$ is the set of world obstacles.

1.2 TODO: Simulated map

TODO: just how it's constructed is relevant, but it's just fixed starified rectangles.

1.3 Updating map from sensors

[SEBA TODO: expand a bit] To navigate the cart-robot system in the environment, an obstacle-free path must be generated. We achieve this in an online fashion, so to account for dynamic obstacles. Before creating a path, obstacles must be identified and converted to a useful representation for the planner. We employ LiDAR scanners to collect distance measurements of the surrounding environment. Then, we convert these distances to a point cloud and morphologically inflate the points into circles in the xy -plane. A morphological union is then performed so to gather circles in bigger blobs, which are then simplified in

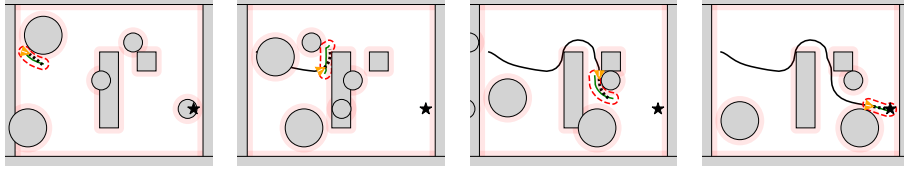


Figure 1: Path generation for a unicycle robot with dynamic obstacles and safety guarantees over time (left to right). The robot (yellow arrow) follows the kinematically constrained trajectory (black dashed) while staying inside the safe area (red boundary) of the local desired trajectory (green solid), reaching the goal (star), never colliding with the dynamic obstacles (circles, translating vertically or horizontally). The overall motion is depicted in solid black, while the obstacle safety boundaries are depicted in light red. Generated with [Dahlin].

the interest of computation time for the next step. The starrification algorithm in [Dahlin] is used to convert the obstacles in *starshaped* obstacles, which are then further inflated with a clearance radius using [Dahlin] to create a *tube-path* that guarantees convergence to the goal pose using the planner in [Huber].